Gunter & Scott 1990 — Inventory of Definitions and Theorems

Source: **C. A. Gunter and D. S. Scott, "Semantic Domains,"** *Handbook of Theoretical Computer Science* Vol. B, North-Holland, 1990, pp. 633–674 (`../papers/Gunter Scott 1990.pdf`).

The work list for the Lean formalization: every definition and every one of the paper's **30 numbered results** (Theorems / Lemmas / Proposition 1–30), in paper order, matched to its Lean equivalent.

Progress (as of r0032, 2026-0806)

#	Quantity	Done	Remaining	Of
1	Definitions to define	**all ≈13** — the computable function landed in r0031; see the note below on `D∞`	0	≈13
2	**Numbered** results complete	**20** (Thm 1, Thm 3, Lem 4, Lem 5, Thm 6, Thm 7, Lem 8, Thm 11, **Thm 12**, Lem 13, Prop 15, Lem 19, Lem 20, Thm 21, Thm 22, Lem 23, Lem 24, Thm 25) — plus Lem 10 and Lem 17, **partially**, see row 2b	9	**29**
2a	**— resolved by refutation**	**Thm 16** — the algebraic-lattice conjunct is proved; the `Fp(D) ↪ (D → D)` embedding conjunct is **false**, kernel-checked (r0032). The result is settled, but it is not a proof of the paper's sentence, so it is counted in neither column	—	—
2b	**— conjunct counts corrected (r0032)**	**Lem 10 is 6 of 7**; **Lem 17 is 5 of 10**. Earlier drafts called both complete. See the note below on the dropped glyphs	—	—
3	**Unnumbered prose claims** proved	**12**	—	—
4	Mathlib foundations reused	12	—	12

#	Quantity	Done	Remaining	Of
5	Theorems in the development	**659** live (+6 commented out as unused)	—	—
6	`sorry` in the development	—	**1**: `thm18` in `Skeleton/Section6.lean`	—

Round r0032. Five agents; three landed, two were still running when this was written.

#	Stream	Landed
1	`Universality.lean`	**Lemma 24** and **Theorem 25**, with `thm25_powerset`: `P N` is universal. Proved at *cpo* strength — no step spends algebraicity or countability, so it is stronger than the paper's statement
2	`IdealCompletion.lean`, `Powerdomain/BoundedComplete.lean`	the **`idealSup` repair**, and bounded completeness reinstated for `D♭` and `D♯`
3	`FinitaryProjectionEmbedding.lean`	**Theorem 16's embedding conjunct refuted**
4	`ContinuousAlgebra.lean`	**Theorem 12** for all three powerdomains, existence *and* uniqueness, 1254 lines, 0 `sorry`
5	`Skeleton/Recovered.lean`, `docs/StatementRecovery.md`	**Lemma 9 and Theorem 14 recovered** from the PDF — both were recorded as "not statable"

`pdftotext` drops glyphs, and four inventory rows undercounted because of it. The paper's 18 embedded Type 3 fonts carry `Custom` encoding with no `ToUnicode` map: codes ≥ `0x20` leak through as ASCII, and **codes below `0x20` vanish silently**. Those are standard TeX positions, pinned by the leaked ASCII (`0x21`→→, `0x3F`→⊥, `0x76`→⊑, all `cmsy10`), so decoding the content stream against `cmsy10`/`cmr10`/`cmmi10` recovers the text character for character; a `pdftocairo` rendering confirms it independently. Consequences:

- **+ and ⊕ are different operators, and what this development has is ⊕.** `lem10_sum` and `lem17_sum` range over `CoalescedSum`, so they discharge the ⊕ conjuncts, not the + ones earlier drafts credited them with. + should be cheap: `D + E = D⊥ ⊕ E⊥`.
- **The strict arrow is two glyphs**, `openbullet` + `arrowright`; both → and ∘→ extract as `!`, so every earlier extraction conflated two function spaces.
- **Lemma 17's three powerdomain conjuncts `D♮, D♯, D♭` were never stated**, though all three powerdomains have existed since r0029.
- **Theorem 12's base theory is `T♮`** (natural), not an undecorated `T`: `pdftotext` renders ♮/♯/♭ as `\`/`]`/`[`, losing the accidental.
- **Two of Lemma 9's conjuncts are misprints** — items 3 and 5 are *false as printed*, refuted on `D = E = Prop`, `F = Prop × Prop` by cardinality (10 vs 8, and 5 vs 3). The corrections are forced, and item 3's is the universal property of ⊕ that the paper states three pages earlier. Recorded in `docs/StatementRecovery.md`; not yet put under the kernel.

Theorem 16's second conjunct is false. The paper asserts the inclusion `i : Fp(D) ↪ (D → D)` is an embedding. Against a five-element bifinite domain with two incomparable minimal upper bounds — one that satisfies `thm16`'s first conjunct, so the refutation isolates the second — there are two incomparable maximal finitary projections below `f = λx. m₁` and no greatest one, while any monotone section would have to produce one. The sketch's error is exact: `p ⊑ f ⟺ im(p) ∩ K(D) ⊆ S_f`, so what is needed is a normal set **contained in** `S_f`, whereas the paper takes the least normal set **containing** it. Those agree only when `S_f` is already normal. No choice of `N_f` repairs it. A positive statement remains available: the conjunct holds when every `S_f` has a greatest normal subposet, which bounded complete domains satisfy.

The `idealSup` repair, closing the third instance of one defect. `idealSup` now branches on `Order.IsIdeal (genIdeal S)` — the ideal *generated by* the union — rather than on the union itself being an ideal. `genIdeal_eq_sUnion_of_isIdeal` proves the two agree wherever the old guard fired, so `lubOfDirected` and `mem_sSup_iff` kept their statements *and their proofs*, and Theorem 11 is byte-identical. `not_boundedComplete_{hoare,smyth,plotkin}` are retired, and `instBoundedCompleteHoare` (no bounded-completeness hypothesis needed) and `instBoundedCompleteSmyth` take their place. The witness is kept as three live lemmas rather than a docstring: the old guard still fails on it, and the repaired `sSup` returns the true least upper bound. A docstring cannot detect a regression.

Round r0029 ran four agents in parallel and closed the definition list.

#	Stream	Landed
1	`Powerdomain/Hoare.lean`	the Hoare (lower) powerdomain: `Pf` the finite non-empty subsets of `K(D)`, the lower pre-order, and `Domain` from Theorem 11
2	`Powerdomain/Smyth.lean`	the Smyth (upper) powerdomain, same shape, dual orientation
3	`Powerdomain/Plotkin.lean`	the Plotkin (convex) powerdomain under the Egli–Milner pre-order
4	`RecursiveDomain.lean`	the recursive domain equation, two formalizations of *universal domain*, and **Theorem 21** — plus `recursiveDomain_funSpace`, the reflexive domain `D ≅ (D → D)`

There is no `D∞` to build. Earlier drafts of this inventory listed `D∞` (inverse limit) as an outstanding definition. Reading §7 directly refutes that: the section raises the chain $T_0 \to_{e_0} T_1 \to_{e_1} T_2 \to \cdots$, says "This is all very informal, however; how are we to make this idea mathematically precise…?", and answers with §7.1, *Solving domain equations with closures*. `D ≅ D → D` is reached from Theorem 21 and Lemma 23, and the limit is taken inside `Fc(U)`. What stands in `D∞`'s place is `Recursive.Solves` / `IsSolvable`, and the two formalizations of universal domain — `IsUniversal` (every domain of the class is a closure of `U`) and `IsUniversalRetract` (every one is a retract), which the paper states as two different sentences.

`Pf` is the finite *non-empty* subsets. The paper defines `Pf(S)` that way and reserves $\bar{P}$ `f(S)` for the version including `∅`; all three powerdomains are built over `Pf`. The distinction is load-bearing and in opposite directions for the three orderings: under the Hoare order `∅ ⊑ v` holds vacuously, so admitting `∅` would add a point strictly below `{⊥}`; under the Smyth order `∅` is a **top**, so it would add a spurious maximum; under Egli–Milner `∅` is comparable to nothing but itself, destroying `OrderBot`. The orchestrator's brief said "finite subsets"; each agent read the PDF and corrected it.

Namespace per agent worked. Every r0029 declaration lives in `ScottDomains.{Hoare, Smyth, Plotkin, Recursive}`. Four agents, **zero** name collisions — against two in r0028, when five agents shared one namespace.

Round r0031 ran four agents and closed the definition list.

#	Stream	Landed
1	`ComputableFunction.lean`	the paper's **computable function**, on Mathlib's `REPred` — the last definition
2	`Powerdomain/BoundedComplete.lean`	**Lemma 13** in the paper's own wording, and a refutation: see the `idealSup` defect below
3	`Powerdomain/Universal.lean`	`isRepresentable_prod` — the product operator is representable over `P N`, **the hypothesis Lemma 24 lacked**; plus `D ≅ D × D`
4	Theorem 18	still running at the time of writing

The `idealSup` defect — the same class, a third time. `IdealCompletion.idealSup` branches its `dite` on `Order.IsIdeal` $(\bigcup_0 \ldots)$. That is the membership condition for the *union*, but the union is the least upper bound only when the family is **directed**; for a merely **bounded** family the least upper bound is the ideal *generated by* the union. So `sSup` returns `⊥` on a bounded non-directed family — kernel-checked in `P N` with the incomparable compacts `{0}` and `{1}`, bounded above by `{0,1}`. This follows `ScottHom` (r0006–r0011) and `Smash` (r0027), and the r0031 plan asserted the guard was correct, which was wrong. Consequently `not_boundedComplete_hoare`, `_smyth` and `_plotkin` are proved: **no `BoundedComplete` instance can be claimed for any ideal completion until `idealSup` is repaired**, by branching on the *generated* ideal rather than on the union being one.

Lemma 13 is nonetheless true and proved, in the paper's own wording (`BddAbove S → ∃ I, IsLUB S I`, with the least element from `OrderBot`): `D♭` (Hoare) needs only the union, so it is bounded complete for **every** domain and the hypothesis is never consumed; `D♯` (Smyth) spends both hypotheses. There is no `lem13_plotkin` to prove — Lemma 13 names only `D]` and `D[`, and Egli–Milner has no join for a bounded pair.

Two inventory rows below are corrected by r0031's reading of §7. Lemmas 28 and 30 are **not** about `P N` and not about `Fc(U)`: both are stated for *p-representability* over `Fp(U)`, Lemma 28 over §7.3's ideal completion of dyadic half-open intervals and Lemma 30 over §7.4's bifinite universal domain. Lemma 23 is therefore **not** Lemma 28's function-space conjunct. Also, the paper proves `F(X) = X + X` has **no** representation over `P N`, so a + conjunct there is false, not missing.

Theorem 12 is recoverable after all. The row below says its "axioms T" are never legible; `pdftotext -layout` extracts them cleanly — a continuous algebra `⊕ : E × E → E` with associativity, commutativity and idempotence, where `T♯` adds `s ⊕ t ⊑ s` and T♭ adds `s ⊑ s ⊕ t`. It belongs to §5.3 with Lemma 13, not to the §7 group.

Round r0028 ran five agents in parallel and roughly doubled the development: 27 → 33 modules, 4440 → 8212 lines, 199 → 384 theorems, 9 → 15 numbered results. Every new result was kernel-audited with `#print axioms`: all depend only on `propext`, `Classical.choice`, `Quot.sound`, and none on `sorryAx`.

#	r0028 stream	Landed
1	`CoalescedSum.lean`, `Skeleton/Sum.lean`	`D + E` as a cpo (`sumSup`, `sumCpo`), then `lem10_sum`, `lem17_sum`, `lem17_smash` — **Lemma 10 at 6 of 6 conjuncts, Lemma 17 at 5 of 5**
2	`IdealCompletion.lean`	**Theorem 11**, both halves, on Mathlib's `Order.Ideal`; §5 unblocked
3	`FinitaryProjectionPoset.lean`, `Skeleton/Section6b.lean`	`Fp(D)` and `Fc(D)` as posets, **Theorem 16** (algebraic-lattice conjunct), **Lemma 20**, and `IsClosure.domain_range` — Lemma 19 at the paper's strength
4	`MinimalUpperBounds.lean`	minimal upper bounds, `U`, `U^∞`, and `isPlotkinOrder_iff_mubClosure` — a characterization the paper does not state. **Theorem 18 still open**
5	`UniversalDomain.lean`	**Theorem 22** and **Lemma 23**, with *representable* defined from §7; opens the route to `D∞`

A name clash `lake build` could not catch. Streams 3 and 5 each defined `IsClosure.apply_sSup_of_directed` and `isClosure_sSup`. The build passed at 971 jobs because no module imported both; the clash appeared the moment anything did — here, an axiom audit importing the pair. `isClosure_sSup` was the *same* statement proved twice; the single copy now lives in `Skeleton/Section6.lean` beside `IsClosure`, which both modules already import, so no call site changed. The two `apply_sSup_of_directed`s were *different* statements sharing a name — one indexed by a set of the subtype `↥(im r)`, one by an ambient `D : Set α` with `D ⊆ im r` — so the subtype form was renamed `IsClosure.apply_sSup_val_image_of_directed`. **A green build is not evidence that parallel work composes**; importing every new module together is.

Row 5 counts lines matching `^(@[…] )?(theorem|lemma)` across the 37 modules.

The `sorry` burn-down (from r0026). The `sorry`s are deliberate scaffolding: fixed *statements* of outstanding results, confined to `ScottDomains/Skeleton/`, one file per agent worktree, so that three agents can prove them in parallel without any agent editing a declaration another depends on. Every other module remains `sorry`-free, and the count above is the burn-down metric — it goes 10 → 0. Round r0027, run as three agents in parallel, took it **10 → 1**.

#	Open statement	Result	State after r0027
1	`prop15`	Prop 15 — every bounded complete domain is bifinite	**proved**
2	`thm18`	Thm 18 — `D`, `D → D` domains ⟹ `D` bifinite	**`sorry`** — the paper gives no proof, citing Smyth [Smy83a]; the obstacle is recorded in the docstring
3	`lem19`	Lem 19 — the image of a closure is a domain	**proved**, via `IsClosure.rangeCompletePartialOrder`
4–7	`lem10_prod`, `lem10_smash`, `lem10_lift`, `lem10_strict`	Lem 10 — bounded completeness closed under `×`, `⊗`, `()⊥`, `→⊥`. The `→` conjunct is **already proved** (Thm 7's bounded-complete half, r0007)	**all four proved**; Lem 10 is then **5 of 6** conjuncts — `D + E` is not stated
8–10	`lem17_prod`, `lem17_lift`, `lem17_fun`	Lem 17 — bifiniteness closed under `×`, `()⊥`, `→`	**all three proved**; Lem 17 is then **3 of 5** conjuncts — `D ⊗ E` and `D + E` are not stated

The + conjuncts, closed in r0028. `CoalescedSum.lean` was 181 lines of ingredients with no `sSup` and no cpo instance, so there was no `D + E` to state a conjunct over. It now has both, and the guard is the membership condition `IsNonBotSum (sumCandidate (sumBase s))` — the defining predicate of the subtype — not directedness, so the defect fixed in `ScottHom` and then in `Smash` did not recur a third time. One wrinkle the smash did not have: `α ⊕ β` carries no `SupSet`, so a summand must be selected first; that selection is not a second guard, because a set with an upper bound at all lies in one summand.

The `smashSup` defect (r0027). `lem10_smash` was not merely open: as `smashSup` stood, it was **false**, and the kernel confirmed a refutation. `smashSup` branched its `dite` on the base being nonempty *and directed*, so a merely **bounded** non-directed base fell to the adjoined `⊥`, which is not even an upper bound. Witness: `D = Prop × Prop`, `E = Prop`, `s = {↑((True, False), True), ↑((False, True), True)}`, bounded by `↑((True, True), True)`. This is the same defect `ScottHom.lean` records having already hit for the function space. The repair branches on the coordinatewise supremum landing in `NonBotPair` — the condition under which it is an element of `D ⊗ E` at all, rather than a merely sufficient condition for it. `smashSup_of_directed` and `smashSup_of_empty` kept their statements and were reproved, which is the agreement claim stated in Lean rather than in prose, and `smashCpo` needed no change.

Kernel check on the r0027 merges (`#print axioms`): all ten proved statements plus `smashCpo`, `smashSup_of_directed` and `smashSup_of_empty` depend only on `propext`, `Classical.choice`, `Quot.sound` — `lem10_prod` and `lem19` do not even need `Classical.choice`. None depends on `sorryAx`. `thm18` does, as its `sorry` requires.

Row 2 counts only the paper's 30 **numbered** results (Theorems / Lemmas / Proposition 1–30). Row 3 counts the claims the paper makes **in prose** rather than as numbered results; these are paper content too, and all twelve are formally verified:

#	Paper claim	Where	Lean
1	"the compact elements [of `P N`] are just the finite subsets of `N`"	p. 9	`isCompactElement_iff_finite`
2	"`P N` … is a domain"	p. 9	`instance : Domain (Set X)`

#	Paper claim	Where	Lean
3	"`D → E` is a … cpo"	Thm 7 proof	`instance : CompletePartialOrder (ScottHom α β)`
4	"… bounded complete … whenever `E` is"	Thm 7 proof	`instance : BoundedComplete (ScottHom α β)`
5	"step(s) … is continuous"	Thm 7 proof	`scottContinuous_stepFun`
6	"… and compact in the ordering on `D → E`"	Thm 7 proof	`isCompactElement_step`
7	"they form a basis for `D → E`"	Thm 7 proof	`instance : IsAlgebraic (ScottHom α β)`
8	every compact function is a *finite* join of step functions	Thm 7 proof, implicit	`exists_finite_isLUB_of_isCo`
9	"an embedding is an injection"	§3.1	`IsEmbeddingProjectionPair.i`
10	"a projection is a surjection"	§3.1	`IsEmbeddingProjectionPair.s`
11	"it is easy to check that `p_N` … is a finitary projection"	§3.1	`isFinitaryProjection_normal`
12	"the set of strict continuous functions `D → E` is also a cpo"	§2.1	`ScottDomains.strictHomCpo`

Six of those eleven are the body of **Theorem 7**, which is now **complete** — all four conjuncts of its conclusion (cpo, bounded complete, algebraic, countably based) are formally verified, as `ScottHom.isBoundedCompleteDomain_scottHom`.

It is proved under **weaker hypotheses than the paper states**: bounded completeness of `D` is never used. `D` need only be a domain. The function space is a cpo for any preordered `D`, algebraic when `D` and `E` are, bounded complete because `E` is, and countably based because `D` and `E` are.

The remaining theorems are supporting API: the ≪ calculus, the `compactsBelow` machinery, the pointwise order and suprema on `D → E`, and the step-function adjunction. The paper assumes or elides all of it.

Reference audit (r0020). Of the theorems in the development, 16 are never cited elsewhere. Nine of those are *terminal by design* — they are the paper's own claims, so nothing should cite them (`injective_embedding`, `surjective_projection`, the `isNormalIn_sUnion*` family and `mono_right` for Lemma 4, `singleton_bot_isNormalIn` for `{⊥} ∈ P(C)`, `isLeast_kleeneFix_le`, `eq_kleeneOperator_op`). The other six were speculative API written for callers that never appeared; they are **commented out in place**, each with a note on why it exists and what is instructive about it, and the build is unchanged — which also confirms that the three `@[simp]` ones among them were never firing implicitly.

The development is **45 modules, 14048 lines, 8 `sorry`, 0 other warnings** (measured by `scripts/counts.sh`). Seven of the eight `sorry`s are the newly *statable* Lemma 9 and Theorem 14 in `Skeleton/Recovered.lean`; the eighth is `thm18`. Counts of definitions, results and theorems are in the Progress table above — they are not repeated here, so that this section cannot drift out of step with it. What each round delivered:

#	Round	Module	Contents
1	r0003	`ScottDomains/WayBelow.lean`	way-below ≪ at `[Preorder α]`, 7 theorems; `x ≪ x ↔ IsCompactElement x` holds by `Iff.rfl`
2	r0004	`ScottDomains/Domain.lean`	`IsAlgebraic`, `Domain` (with the paper's countable-basis condition), `BoundedComplete`, 8 theorems, `Domain Prop`
3	r0005	`ScottDomains/Powerset.lean`	the paper's `P N` (p. 9): compacts of `Set X` are exactly the finite subsets, hence `Domain (Set ℕ)` — the nondegenerate witness
4	r0006–r0007	`ScottDomains/ScottHom.lean`	the continuous function space `D → E`: `ScottHom`, the pointwise order, `CompletePartialOrder`, and `BoundedComplete` when `E` is — **Theorem 7's first sentence in full**

#	Round	Module	Contents
5	r0008	`ScottDomains/StepFunction.lean`	the single step function `step k e`: continuity (from `k` compact), the adjunction `step k e ≤ f ↔ e ≤ f k`, and compactness in `D → E` (from `e` compact)
6	r0009	`ScottDomains/FunctionSpaceDomain.lean`	**`D → E` is algebraic** — the paper's "they form a basis for `D → E`"; the two halves of `IsAlgebraic` use disjoint hypotheses (directedness needs only `E` bounded complete; the lub needs only `D`, `E` algebraic)
7	r0010	`ScottDomains/CompactFunction.lean`	every compact function is a **finite** join of step functions — the finiteness half of the basis claim
8	r0011	`ScottDomains/FunctionSpaceCountable.lean`	`K(D → E)` is countable, hence `Domain (ScottHom α β)` — **Theorem 7 complete**
9	r0012	`ScottDomains/NormalSubposet.lean`	the normal-subposet relation ◁ and **Lemma 4**, all four parts
10	r0012–r0013	`ScottDomains/Projection.lean`	embedding–projection pairs, projections, the paper's "an embedding is an injection, a projection is a surjection"; **`im(p)` is a cpo** and **finitary projections**
11	r0014	`ScottDomains/FinitaryProjection.lean`	**Lemma 5** — the compacts of `im(p)` are `im(p) ∩ K(D)` (needs only that `p` is a projection), and `im(p) ∩ K(D) ◁ K(D)`
12	r0015	`ScottDomains/NormalProjection.lean`	`p_N(x) = ⨆{y ∈ N | y ⊑ x}`: continuous, a projection, and `im(p_N) ∩ K(D) = N` — half of **Theorem 6**'s correspondence, plus order preservation both ways
13	r0016	`ScottDomains/Theorem6.lean`	**Theorem 6** — `p_N` is finitary, `p_{im(p) ∩ K(D)} = p`, and the correspondence assembled
14	r0017	`ScottDomains/FixedPoint.lean`	**Theorem 1** — `⨆ₙ fⁿ(⊥)` is the least fixed point of a continuous `f` on a cpo. Not Mathlib reuse; see the §2 table
15	r0018	`ScottDomains/UniformFixedPoint.lean`	**Theorem 3** — `fix` is the unique uniform fixed-point operator; `↓a` as a cpo
16	r0019	`ScottDomains/Product.lean`	`D × E` as a cpo (the one construction needing no case split) and **Lemma 8 parts 1–3**
17	r0021	`ScottDomains/Currying.lean`	**Lemma 8.4** — currying, `D → (E → F) ≅ (D × E) → F`, and with it **Lemma 8 complete**
18	r0022	`ScottDomains/EffectivePresentation.lean`	§3.2's **effective presentation** — the enumeration of the basis with its two decidability conditions
19	r0023	`ScottDomains/Lift.lean`	the **lift** `D⊥` as a cpo, on Mathlib's `WithBot`
20	r0024	`ScottDomains/StrictHom.lean`	the **strict function space** `D →⊥ E` as a cpo — needs no case split, since both branches of `ScottHom`'s `sSup` are strict
21	r0025	`ScottDomains/Smash.lean`	§4.3's **smash product** `D ⊗ E` — the non-bottom pairs with a new bottom adjoined, as a cpo
22	r0025	`ScottDomains/Bifinite.lean`	§6.1's **Plotkin order** and **bifinite** domain

§2 and §3 are now complete — the only §3 omission is the paper's *computable function*, which needs an r.e.-predicate notion Mathlib does not supply.

Dependency structure of the remaining definitions

#	Definition	Prerequisites	Status
1	smash product `D ⊗ E`	`Domain`, `OrderBot`	**✓ done, r0025**
2	bifinite / Plotkin order	`IsNormalIn` (◁, r0012)	**✓ done, r0025**
3	sum `D + E`	`Domain`	independent, ready
4	`D∞`	embedding–projection pairs (r0012)	independent, but large (inverse limits)
5	the three powerdomains	the **ideal completion** (Theorem 11) — *not* built	**blocked**; the three are independent of each other once it exists

Rows 1 and 2 were written as two modules and checked in a single build, which is the parallelism the structure actually admits — the bottleneck is the build-and-fix loop, not the writing.

Next: the sum `D + E`, then Theorem 11 to unblock the powerdomains, then Lemma 9 (the strict analogues of Lemma 8) and Lemma 10 (closure of bounded completeness).

Work counts

- **Reuse from Mathlib — no work (11):** poset, directed, cpo, ⊥, monotone, continuous, fixed-point operator, Schröder–Bernstein (2), algebraic lattice, product, λ-notation. (Theorem 1 was listed here in an earlier draft and has been removed: Mathlib's `OrderHom.lfp` is Knaster–Tarski over a complete lattice, not Kleene's $\bigsqcup_n f^n(\bot)$ over a cpo. It is proved in `ScottDomains/FixedPoint.lean`, so **29** numbered results needed proof, not 28.)
- **Generalize / adapt (4):** compact element (`IsCompactElement` — its *definition* is already stated at `[PartialOrder α]` and so applies to a dcpo verbatim; what is `CompleteLattice`-only is every lemma about it, so the dcpo API must be re-proved), sum, lift (`WithBot`/`Sum`), ideal completion (`Order.Ideal`).
- **Definitions to define — new (≈13; 4 done in r0003–r0004, 9 remaining):** way-below ≪ (**done** — `ScottDomains/WayBelow.lean`), algebraic cpo, **domain**, bounded-complete (**all three done** — `ScottDomains/Domain.lean`), embedding–projection pair, (finitary) projection, normal subposet, effective presentation, smash product, the three powerdomains (Hoare / Smyth / Plotkin), bifinite / Plotkin order, and `D∞`.
- **Theorems to prove (28):** 28 of the paper's 30 numbered results — Theorems 3, 6, 7, 11, 12, 14, 16, 18, 21, 22, 25, 26, 27, 29; Lemmas 4, 5, 8–10, 13, 17, 19, 20, 23, 24, 28, 30; Proposition 15. Only Theorems 1 & 2 come free from Mathlib.

Bottom line: ≈13 definitions to define + 28 results to prove, on top of 12 reused Mathlib foundations. After r0004: **9 definitions + 28 results** remain.

Lean column legend: ✓ reuse Mathlib (name given) · ~ partial (Mathlib has a related or lattice-only version — generalize) · ✗ define / prove (absent from Mathlib v4.32.2, confirmed by grep).

> Statements are paraphrased/de-garbled from the PDF (1990 Type-3 fonts render → as `!`, drop `fi` ligatures, mangle math), so read them as a guide to *what* each result is. Notation: ⊑ order, ⨆ directed sup, ⊥ bottom, ≪ way-below, `K(D)` compacts, `Fp(D)` finitary projections.

§2 Recursive definitions of functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
2.1	Def	—	**poset** (partially ordered set)	✓ `PartialOrder`
2.1	Def	—	**directed** subset `M`: every finite `u ⊆ M` has an upper bound in `M`	✓ `DirectedOn` / `IsDirected`
2.1	Def	—	**cpo**: poset in which every directed `M` has a lub ⨆`M`	✓ `CompletePartialOrder` (chains: `OmegaCompletePartialOrder`)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
2.1	Def	—	**bottom** `⊥` (least element)	✓ `OrderBot`
2.1	Def	—	**monotone** function	✓ `Monotone` / `OrderHom`
2.1	Def	—	**continuous**: monotone, `f(⨆M) = ⨆f(M)` for directed `M`	✓ `ScottContinuous`
2.1	Thm	1	**Fixed-Point Theorem**: `f : D → D` continuous ⟹ least fixed point `⨆_n fⁿ(⊥)`	✓ **proved** (r0017) — `ScottDomains.theorem1`. **Not** Mathlib reuse: `OrderHom.lfp` is Knaster–Tarski over a *complete lattice* with only monotonicity, a different theorem
2.2	Thm	2	**Schröder–Bernstein** for sets	✓ `Function.schroeder_bernstein` (`SetTheory/Cardinal/SchroederBernstein.lean`) — the name in an earlier draft of this row, `Function.Embedding.schroederBernstein`, does not exist
2.3	Def	—	**fixed-point operator** (uniform)	✓ `OrderHom.lfp` / `LawfulFix`
2.3	Thm	3	The standard operator is the **unique uniform** fixed-point operator	✓ **proved** (r0018) — `ScottDomains.theorem3`; a fixed point operator is formalized as a family over every `CompletePartialOrder` in a universe

§3 Effectively presented domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	**compact element** `x`: `x ⊑ ⨆M` (dir.) ⟹ `x ⊑ y` some `y∈M`; `K(D)`	~ `IsCompactElement` — def is `[PartialOrder]`, so reusable on a dcpo; its lemmas are all `CompleteLattice`-only
3.1	Def	—	**way-below** ≪ (approximation relation behind compactness)	✓ `ScottDomains.WayBelow` (r0003) — `x ≪ x ↔ IsCompactElement x` by `Iff.rfl`
3.1	Def	—	**algebraic** cpo: `x = ⨆{x'∈K(D) : x'⊑x}` (directed)	✓ `ScottDomains.IsAlgebraic` (r0004)
3.1	Def	—	**domain** = algebraic cpo **whose basis `K(D)` is countable** (the paper's definition, p. 9 — the countability condition was missing from an earlier draft of this row)	✓ `ScottDomains.Domain` (r0004)
3.1	Def	—	**bounded complete**: `⊥` + every bounded subset has a sup — a *separate* predicate; the paper composes them as "bounded complete domain" (Thm 7, Lem 10, Lem 13, Thm 14), which is the literature's *Scott domain*	✓ `ScottDomains.BoundedComplete` (r0004); the compound is `[Domain α] [BoundedComplete α]`
3.1	Def	—	**(countably based) algebraic lattice**	✓ `IsCompactlyGenerated` (+`CompleteLattice`)
3.1	Def	—	**embedding–projection pair** `(g, f)`	✓ `ScottHom.IsEmbeddingProjectionPair` (r0012)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	**projection**; **finitary projection** `p`: `p∘p=p⊑id`, `im(p)` a domain	✓ `ScottHom.IsProjection` (r0012), `ScottHom.IsFinitaryProjection` (r0013) — `im(p)` carries a `CompletePartialOrder` via `IsProjection.rangeCompletePartialOrder`
3.1	Def	—	**normal subposet** / substructure	✓ `ScottDomains.IsNormalIn`, notation ◁ (r0012)
3.1	Lem	4	`⟨P(C), ◁⟩` of substructures is a cpo with `{⊥}` least	✓ **proved** (r0012), all four parts
3.1	Lem	5	`p` finitary projection ⟹ compacts of `im(p)` are `im(p) ∩ K(D)`, and `im(p) ∩ K(D) ◁ K(D)`	✓ **proved** (r0014) — `IsProjection.isCompactElement_iff` (needs only *projection*) and `IsFinitaryProjection.isNormalIn_compacts`
3.1	Thm	6	Isomorphism: normal substructures ≅ `Fp(D)` (finitary projections)	✓ **proved** (r0015–r0016) — `ScottDomains.theorem6`
3.1	Thm	7	`D,E` bounded-complete domains ⟹ `D → E` bounded-complete domain	✓ **proved** (r0006–r0011) — `ScottHom.isBoundedCompleteDomain_scottHom`; `D` bounded complete is not needed
3.2	Def	—	**effective presentation** `d : ℕ → K(D)`; **effectively presented domain**	✓ `ScottDomains.EffectivePresentation` (r0022). The paper's **computable function** is ✓ **defined** (r0031) in `ComputableFunction.lean`, on Mathlib's **`REPred`** (`Mathlib/Computability/RE.lean:157`). An earlier draft of this row said "Mathlib v4.32.2 has no `RePred` or equivalent (grep finds none)" — that grep used the wrong capitalization, and the claim was false. Two caveats recorded there: `decidableLE` is too weak to prove anything computable (a `Decidable` instance may be `Classical.dec`), so results need an explicit `RecursiveLE` hypothesis; and composition awaits `REPred` closure under `∧` and `∃`, which Mathlib's five-lemma API does not supply

§4 Operators and functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.1	Def	—	**product** `D × E`	✓ `Prod` order from Mathlib; the **cpo instance** is `ScottDomains.instCompletePartialOrderProd` (r0019) — Mathlib has `Prod.supSet` and `isLUB_prod` but no cpo instance
4.2	Def	—	**Church's λ-notation** (continuous abstraction)	✓ `OrderHom` / ωCPO `ContinuousHom`
4.3	Def	—	**smash product** `D ⊗ E`	✓ `ScottDomains.Smash` + `smashCpo` (r0025)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.4	Def	—	**sum** `D + E`; **lift** `D⊥`	both ✓ — lift `ScottDomains.liftCpo` on `WithBot` (r0023); the coalesced sum `CoalescedSum` with `sumSup` and `sumCpo` (r0028), its `sSup` guarded on landing in `NonBotSum`
4.x	Lem	8	`D×E ≅ E×D`; `(D×E)×F ≅ D×(E×F)`; `D→(E×F) ≅ (D→E)×(D→F)`; `D→(E→F) ≅ (D×E)→F`	✓ **proved** — `prodComm`, `prodAssoc`, `scottHomProd` (r0019); `scottHomCurry` (r0021)
4.x	Lem	9	Iso laws over `D,E,F`: `D⊗E ≅ E⊗D`; `(D⊗E)⊗F ≅ D⊗(E⊗F)`; `(E⊕F) ∘→ D ≅ (E ∘→ D) × (F ∘→ D)`; `D ∘→ (E ∘→ F) ≅ (D⊗E) ∘→ F`; `D⊗(E⊕F) ≅ (D⊗E)⊕(D⊗F)`; `D⊥ ∘→ E ≅ D → E`	✗ prove — **statements recovered** (r0032, `Skeleton/Recovered.lean`, 6 `sorry`). Four are certain, two are inferred **corrections of misprints**: items 3 and 5 are false as printed, refuted by cardinality on `D = E = Prop`, `F = Prop × Prop`. `∘→` is the strict function space; it and `→` both extract as `!`
4.5	Lem	10	`D,E` bounded complete ⟹ `→, ∘→, ×, ⊗, ⊕, +, ()⊥` bounded complete	~ **6 of 7 conjuncts** — `→` r0007; `×`, `⊗`, `()⊥`, `∘→` r0027 (`Skeleton/Lemma10.lean`); `⊕` r0028 (`Skeleton/Sum.lean`, over `CoalescedSum`). **+ is not proved**: it is a different operator from `⊕`, and should be cheap via `D + E = D⊥ ⊕ E⊥`
4.5	Thm	11	**Ideal completion** of a countable pre-order is a domain (all domains so arise)	✓ **proved** (r0028) — `IdealCompletion.thm11` and `thm11_converse`, on Mathlib's `Order.Ideal`
4.5	Thm	12	Initiality of a continuous algebra satisfying the axioms **`T♮`** (natural — not an undecorated `T`; `pdftotext` renders `♮/♯/♭` as `\/]/[`)	✓ **proved** (r0032) — `ContinuousAlgebra.thm12_plotkin`, `thm12_smyth`, `thm12_hoare`: `T♮ → D♮`, `T♯ → D♯`, `T♭ → D♭`, each `∃! h`, existence *and* uniqueness, factoring through `{|·|}` rather than the weaker principal ideals. `[IsAlgebraic D]` is the whole hypothesis — countability of `K(D)` is never used, and bounded completeness is never needed. Each free algebra is also proved a model of its own theory, a gap the plan had not named
4.5	Lem	13	`D` bounded complete ⟹ powerdomains `D]`,`D[` bounded complete	✓ **proved** (r0031) — `PowerdomainBC.lem13_hoare`, `lem13_smyth`, in the paper's wording. `D♭` needs no bounded-completeness hypothesis at all; there is no convex conjunct to prove

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.5	Thm	14	Equivalent characterizations of an (algebraic/BC) domain	✗ prove — **statement recovered** (r0032, `Skeleton/Recovered.lean`, 1 `sorry`). The text was never garbled; only the `fi` ligature drops. The real blocker was that `Bifinite.lean` *defines* `IsBifinite` as condition 2, making the theorem `P ↔ P`; `IsBifiniteViaProjections` supplies condition 1 from the paper's own definition, and `thm14` is the equivalence — the result that licenses §6's use of the Plotkin-order condition as the definition

§5 Powerdomains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.1	Def	—	**powerdomain** (non-deterministic outcomes)	✓ each of the three is `IdealCompletion (Pf K(D))` under its pre-order (r0029)
5.2	Def	—	**Hoare (lower)**, **Smyth (upper)**, **Plotkin (convex)** powerdomains	✓ `ScottDomains.Hoare.Powerdomain`, `Smyth.Powerdomain`, `Plotkin.Powerdomain` (r0029), each with its `Domain` instance from Theorem 11 and its compacts characterized as the principal ideals
5.3	—	—	Universal & closure properties (see Lem 13, 28, 30)	✗ prove — r0030 wave A

§6 Bifinite (SFP) domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.1	Def	—	**Plotkin order** / **bifinite (SFP) domain**	✓ `ScottDomains.IsPlotkinOrder`, `IsBifinite` (r0025)
6.1	Prop	15	Every bounded-complete domain is bifinite	✓ **proved** (r0027) — `ScottDomains.prop15`, the paper's own proof over `lubClosure u`
6.1	Thm	16	`D` bifinite $\Longrightarrow$ `Fp(D)` is an algebraic lattice	**settled, half each way**: the algebraic-lattice conjunct ✓ **proved** (r0028) as `ScottDomains.thm16`; the `Fp(D) ↪ (D → D)` embedding conjunct ✗ **refuted** (r0032) in `FinitaryProjectionEmbedding.lean`, kernel-checked, with the sketch's exact error identified — see the note under Progress

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.2	Lem	17	`D`,`E` bifinite $\Longrightarrow$ `→, ∘→, ×, ⊗, ⊕, +, ()⊥, D♮, D♯, D♭` bifinite	~ **5 of 10 conjuncts** — `×`, `()⊥`, `→` r0027 (`Skeleton/Lemma17.lean`); `⊗`, `⊕` r0028 (`Skeleton/Sum.lean`). **Not proved:** `∘→`, `+`, and the three powerdomains `D♮`, `D♯`, `D♭` — the last three were dropped from the extraction with their glyphs, though all three powerdomains have existed since r0029
6.2	Thm	18	If `D` and `D → D` are domains, then `D` is bifinite	✗ prove — **the development's only `sorry`**. Reduced by `isBifinite_iff_mubClosure` (r0028) to two obligations; blocked on a constructor for continuous functions on a domain that is not bounded complete
6.2	Lem	19	closure `r:D→D` (`r∘r=r⊒id`) $\Longrightarrow$ `im(r)` is a domain	✓ **proved** — `lem19` (r0027) gives the cpo structure; `IsClosure.domain_range` (r0028) gives the paper's full strength, `im(r)` a domain with basis `{r(k) | k ∈ K(D)}`
6.2	Lem	20	`D` domain $\Longrightarrow$ `Fc(D)` (finitary closures) is a cpo	✓ **proved** (r0028) — `ScottDomains.lem20`, over `Fc α` with the pointwise order

§7 Recursive definitions of domains (universal domain, `D∞`)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Def	—	**recursive domain equation**; **universal domain**. (Earlier drafts of this row said "`D∞` (inverse limit)" — §7 builds no inverse limit; see the note under Progress)	✓ `Recursive.Solves` / `IsSolvable`, and `Recursive.IsUniversal` / `IsUniversalRetract` for the paper's two phrasings (r0029)
7	Thm	21	`F` representable over cpo `U` $\Longrightarrow$ a domain `D` with `D ≅ F(D)`	✓ **proved** (r0029) — `ScottDomains.Recursive.thm21`; with `IsRepresentable₂.diag` and Lemma 23 it yields `recursiveDomain_funSpace`, the reflexive domain `D ≅ (D → D)`
7	Thm	22	Any countably-based algebraic lattice `L`: a closure `r : P(ℕ) → L`	✓ **proved** (r0028) — `ScottDomains.thm22`, with `thm22_of_isCompactlyGenerated` the Mathlib-vocabulary form
7	Lem	23	The function-space operator is representable over `P(ℕ)`	✓ **proved** (r0028) — `ScottDomains.lem23`
7	Lem	24	`U` cpo; `×` and `→` representable over `U` $\Longrightarrow$ (setup for universality)	✓ **proved** (r0032) — `Universality.lem24`, concluding also that `D`, `E` are closures of `U`, which its own proof needs
7	Thm	25	`U` non-trivial domain representing `×`,`→` $\Longrightarrow$ `U` **universal**	✓ **proved** (r0032) — `Universality.thm25`, with `thm25_powerset` instantiating it at `P N` and `thm25_isUniversal` in `Recursive.IsUniversal` form. Proved at **cpo** strength: no step spends algebraicity or countability, so it is stronger than the paper's statement

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Thm	26	Any signature `(s₁,…,s_n)`: combinators `F₁,…,F_n` solving the equations	✗ prove
7	Thm	27	Any bounded-complete `D`: a projection of the universal domain onto `D`	✗ prove
7	Lem	28	Operators `→,×,⊗,+,()⊥,()],()[` representable over `U`	✗ prove
7	Thm	29	`D` bifinite ⟹ `D+` bifinite; solving `D ≅ D+`	✗ prove
7	Lem	30	Operators `→,×,⊗,+,()⊥,()],()[` p-representable over universal bifinite `V`	✗ prove

---

Tally (matched): ✓ reuse Mathlib ≈ **12** (poset, directed, cpo, ⊥, monotone, continuous, fixed-point Thm 1 & operator, Schröder–Bernstein, algebraic lattice, product, λ-notation) · ~ partial ≈ **4** (compact element, sum/lift, ideal completion Thm 11, Thm 22) · ✗ define / prove ≈ **44**, of which 4 are done (≪ r0003; algebraic, domain, bounded-complete r0004) → **40 remaining** — the bifinite / powerdomain / `D∞` development and all 28 numbered results.

What is done, and what is next, is listed under **Progress** at the top of this file.